

Hello everyone. This is Part 2 of BLA 3 in my AWS Certified Machine Learning Engineer Associate MLA-C02 learning series. Part 1 focused on choosing an appropriate model or managed AI service. This presentation explains what happens after that choice.

I will discuss the training process, SageMaker AI training options, important hyperparameters, overfitting and underfitting, methods for reducing training time, and different ways to customize foundation models.

A typical training workflow begins with configuration. The engineer selects the algorithm, framework, compute resources, input data, target metric, and important hyperparameters.

During training, the algorithm learns patterns from the training data. Validation data provides an independent view of performance while the team adjusts the model.

Tuning compares different settings, while experiment tracking records what changed and what result each run produced.

The team should also define stopping conditions and failure handling. Training may stop because the metric stopped improving, resources failed, or a cost limit was reached. A repeatable workflow makes troubleshooting easier and allows another person to reproduce the model.

Amazon SageMaker AI supports several ways to train machine learning models. Built-in algorithms provide managed implementations for common problems such as classification, regression, clustering, and anomaly detection.

Script mode allows a team to bring training code written with supported frameworks such as PyTorch, TensorFlow, or scikit-learn. SageMaker manages the training environment while the script defines the model and learning process.

Custom containers provide the greatest control. A team can package special libraries, dependencies, or training tools into a container.

The selection depends on the amount of customization required.

Hyperparameters are settings chosen before or during training rather than values learned directly from the data.

An epoch represents a complete pass through the training data. Batch size controls how many examples the model processes before updating its parameters.

SageMaker automatic model tuning can run multiple training jobs with different hyperparameter combinations.

Automatic tuning does not remove the need for judgment. The metric must represent the business objective, the search space must be reasonable, and the final model still

needs an independent evaluation. Tuning should improve a clear baseline rather than search without a defined goal.

Underfitting occurs when a model has not learned enough from the training data. It performs poorly on both training and validation examples. The model may be too simple, the features may not contain useful information, the training time may be too short, or the learning settings may be unsuitable.

Overfitting occurs when the model learns the training data too closely. Training performance looks strong, but validation performance is weaker. The model may have too much capacity, limited training data, noisy labels, or leakage. Possible responses include regularization, simpler architecture, early stopping, more representative data, or data augmentation.

Training efficiency matters because large datasets and models can consume significant time and compute. Early stopping ends training when the validation metric no longer improves.

Distributed training uses multiple computing resources. Data parallelism gives different batches to multiple workers, while model parallelism divides a model that is too large for one device.

Checkpoints save the state of training at selected points. If a job stops, the team may continue from a checkpoint instead of starting again.

Foundation-model customization can begin with prompt design. Clear instructions, examples, output formats, and restrictions often improve results without changing model weights. Prompt management helps teams version, test, and reuse prompts consistently.

RAG adds external knowledge at request time.

Fine-tuning changes model behavior using training examples.

The team should compare these approaches using the same evaluation set.

To conclude Part 2, training should produce both a model and a clear record of how that model was created. The team should version the training data, validation data, code, prompts, configuration, and model artifact. Hyperparameters, metrics, compute resources, and random seeds should also be recorded.

My main takeaway is that better training does not always mean more training. A controlled process should improve the required metric while respecting time and cost. In Part 3, I will discuss model evaluation, explainability, human review, drift, shadow testing, and monitoring for traditional machine learning and generative AI.

